

Ideation Report

AI-Powered Renewable Generation Forecasting Platform

Team Name: The Final Commit

Event: HackOut 2026, organized by Synapse

College: Dhirubhai Ambani University

Theme: Renewable Energy Intelligence

Members: Dhruvil Mehta, Kashyap Ajudiya, Aaryan Sangani, Banshi Mehta

1. Problem Statement

Solar and wind power output changes constantly with weather, time of day, and season. This makes it hard for grid operators and utilities to plan capacity, schedule backup power, or avoid curtailment. We are building an intelligence platform that ingests weather data, historical generation records, and site level parameters to forecast solar and wind output for the next 24 to 72 hours, flag periods of expected over or under generation, and recommend grid actions such as curtailment, storage dispatch, or backup activation.

2. Why This Matters

Renewable energy is intermittent by nature. Without a reliable forecast, grid operators over provision fossil fuel backup as a safety margin, and plant owners lose revenue to unplanned curtailment when the grid cannot absorb excess generation. A good forecasting and decision system directly addresses three real problems:

- **Wastage and emissions:** Accurate forecasts reduce unnecessary fossil fuel backup and cut down curtailed clean energy that goes to waste.
- **Grid stability:** Knowing supply fluctuations 24 to 72 hours in advance lets operators plan reserves and switching instead of reacting at the last minute.
- **Financial planning:** Energy traders and plant owners can bid into markets and plan maintenance windows with more confidence when output is predictable.

This matters beyond the hackathon because renewable capacity on grids is growing faster than the forecasting infrastructure supporting it, especially at the regional and plant level in India.

3. Proposed Solution

We propose a three layer system: a **forecasting layer** that predicts solar and wind generation using weather and historical data, a **decision layer** that converts forecasts into concrete grid actions, and a **dashboard layer** that presents forecasts, alerts, and recommendations to grid operators and traders in real time. The decision layer is what turns a forecasting model into a genuinely useful operational tool, since the problem statement explicitly asks for action recommendations, not just predicted numbers.

4. Key Features

- **24 to 72 hour generation forecast** for solar and wind, per site, updated as new weather data arrives.
- **Confidence intervals on every forecast** so operators know how much to trust a given prediction, not just a single number.
- **Over/under generation flags:** automatic detection of periods where forecast supply deviates significantly from expected demand or grid capacity.
- **Action recommendation engine:** suggests curtailment, storage dispatch, or backup activation based on the size and direction of the deviation.

- **Real time dashboard** with generation trends, weather overlays, alerts, and a site comparison view.
- **Explainability:** feature importance shown alongside each forecast (for example, cloud cover or wind speed driving a dip) so operators trust the recommendation.

5. System Architecture

Data flows from external sources into a pipeline that cleans and joins it, feeds a forecasting model, passes the forecast to a rule based decision engine, and finally surfaces everything on the dashboard.

1. Weather/satellite APIs + historical generation data
2. Data pipeline (ingestion, cleaning, feature store)
3. Forecasting model (baseline + XGBoost, optional deep learning)
4. Decision engine (thresholds on forecast vs capacity/demand)
5. Dashboard and alerts (React + D3.js, backend API)

Each stage is intentionally decoupled so the model and decision logic can be improved later without redesigning the pipeline or the dashboard.

6. Data Sources

Source	Type	Why we chose it
NREL NSRDB / PVGIS	Solar irradiance, historical	Free, high quality solar radiation data used widely to benchmark solar models.
Open-Meteo / NOAA GFS / ECMWF	Weather forecast API	Gives the 24 to 72 hour forward looking weather variables our forecast actually depends on.
NIWE / Grid-India (POSOCO) reports	Wind resource + grid generation (India)	Makes the platform relevant to Indian grid conditions, not just generic global data.
Kaggle solar/wind plant datasets	Historical plant-level generation	Quick to access for prototyping and model validation before integrating live feeds.
Site metadata (self-defined)	Panel tilt, capacity, turbine specs	Generation depends on site hardware, not just weather; needed so the model generalizes across sites.

7. Our Approach and Roadmap

We are building this in stages, starting simple and adding complexity only where it earns its place. Each step below includes why we are doing it, not just what we are doing.

Step 1: Data Collection

What: Pull historical solar/wind generation and matching weather records, and set up a live weather API feed.

Why: We cannot forecast what we have not measured. Historical data lets us train and validate; the live feed is what makes 24 to 72 hour forecasts possible at all, since forecast weather is the main driver of near term generation.

Step 2: Data Preprocessing

What: Clean missing/faulty readings, align timestamps across sources, resample to a consistent interval, and handle site-specific gaps.

Why: Weather and generation data come from different sources with different time resolutions. Misaligned or dirty data silently corrupts every model built on top of it, so this step protects everything downstream.

Step 3: Feature Engineering

What: Derive features such as time of day, solar elevation angle, cloud cover trend, wind shear, lag features (past generation), rolling averages, and site capacity/orientation.

Why: Raw weather values alone underperform. Physically informed features (like solar angle or wind shear) give the model information closer to what actually drives output, which improves accuracy without needing a bigger model.

Step 4: Baseline Model (Linear Regression)

What: Train a simple linear regression per site as a first benchmark.

Why: A baseline tells us the floor. If a complex model cannot beat linear regression by a meaningful margin, that is a signal to fix data or features before adding model complexity, not to make the model bigger.

Step 5: Main Model (XGBoost or LightGBM)

What: Train gradient boosted trees on the engineered features, tuned per site or per site cluster. We will evaluate both XGBoost and LightGBM on our actual data and finalize one before the submission deadline.

Why: Both handle non-linear weather-to-generation relationships well and give feature importance out of the box, which we need for explainability on the dashboard. LightGBM is generally faster and handles many features/large data more efficiently, which helps with repeated retraining, while XGBoost tends to be more stable on smaller per-site datasets with less tuning. Since we don't yet know our final dataset size per site, we are keeping both in play and will pick whichever performs better in practice rather than committing upfront.

Step 6: Evaluation and Uncertainty Quantification

What: Evaluate with MAPE, RMSE, and MAE against a held-out time period, and generate prediction intervals (e.g. quantile regression) alongside point forecasts.

Why: Grid operators do not just need a number, they need to know how much to trust it. A forecast with a confidence band supports risk-based decisions far better than a single point value.

Step 7: Decision Engine

What: Build a rule-based layer that compares forecast generation against demand/grid capacity thresholds and outputs a recommendation: curtail, dispatch storage, or activate backup.

Why: This is the part of the problem statement that turns a forecasting tool into a decision support tool. Without it we only answer 'what will happen', not 'what should we do about it', which is what grid operators actually need.

Step 8: Dashboard (Frontend + Backend)

What: Build a real-time dashboard in React with D3.js visualizations, backed by an API serving forecasts, flags, and recommendations.

Why: The best model is not useful if nobody can act on it quickly. A clear dashboard with alerts is what actually gets used by operators and traders during a shift.

Step 9: Cloud Pipeline and Scalability

What: Containerize the pipeline and schedule periodic retraining and re-forecasting jobs (e.g. every few hours as new weather data arrives).

Why: A forecast is only useful if it stays current. Scheduling ensures the system keeps working after the demo, and containerization makes it easy to add more sites later.

Step 10 (Stretch Goal): Fine-tune a Transformer/LSTM

What: If time permits, fine-tune a sequence model (e.g. LSTM or a lightweight transformer) on the same features and compare it against XGBoost.

Why: Deep sequence models can capture longer temporal patterns than tree models, but they need more data and tuning time. We treat this as a stretch goal so a working, explainable system is guaranteed first, with a performance upgrade attempted only if the core pipeline is solid.

8. Technology Stack

Layer	Technology	Purpose
Data & Modeling	Python, Pandas, scikit-learn, XGBoost / LightGBM (final choice after evaluation), Prophet (comparison), PyTorch (stretch goal)	Data processing and forecasting models
Weather Data	Open-Meteo / NOAA GFS / NREL NSRDB APIs	Historical and forecast weather inputs
Backend	FastAPI / Flask, PostgreSQL	Serving forecasts and recommendations to the dashboard
Frontend	React, D3.js, Recharts	Real time dashboard and visualizations
Infra	Docker, cloud scheduler (cron/Airflow-lite)	Automated retraining and periodic re-forecasting

9. Expected Impact

- Lower reliance on fossil fuel backup plants through accurate advance forecasts.
- Fewer curtailment events, meaning less clean energy wasted and better revenue for plant owners.
- More stable grid operation since supply fluctuations are anticipated instead of reacted to.
- A reusable, site-agnostic pipeline that can scale from a few plants to a regional grid.

10. Future Scope

- Extend the decision engine with real market price signals for smarter storage/trading recommendations.
- Add satellite imagery based nowcasting for very short-term (sub-hour) accuracy.
- Support multi-region grid views for utilities managing many sites at once.
- Replace rule-based thresholds in the decision engine with a learned policy once enough operational feedback is collected.